SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*
(BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20 Total Marks:20 Duration :1 Hour

Mock Interview question

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action. ***Signature of the student:*** 

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?
2. Define the components of a PDA formally.
3. What is a stack in PDA? Why is it important?
4. What is an Instantaneous Description (ID) in PDA?
5. What are the different types of moves in a PDA?
6. What is acceptance by final state and empty stack?
7. Define the language accepted by a PDA.
8. How does a PDA process a string using IDs?
9. Differentiate between Deterministic PDA and Non-Deterministic PDA.
10. Why are PDAs equivalent to Context-Free Grammars (CFG)?
11. What types of languages are accepted by PDA? Give examples.
12. What is a Turing Machine? How is it more powerful than PDA?
13. Define the components of a Turing Machine.
14. Explain tape, head, and state transitions in a Turing Machine.
15. Explain programming techniques for Turing Machines.
16. What is storage in finite control in Turing Machines?
17. Compare FA, PDA, and TM based on memory, power, and language acceptance. 18. What is DFA?
19. Which language is accepted by a Turing Machine?
20. How turing machines are applied in real world applications?

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application

Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

Interview Answer

A Pushdown Automaton, or PDA, is a finite automaton with an additional stack. The stack gives the PDA extra memory, so it can recognize many context-free languages.

A Finite Automaton has only a finite number of states and no stack. Therefore, it can recognize only regular languages. A PDA has a stack and can recognize context-free languages such as $\{a^n b^n \mid n \geq 1\}$.

Key Points to Remember

- FA has finite control only.
- PDA has finite control plus one stack.
- FA recognizes regular languages.
- PDA recognizes context-free languages.
- Example: $a^n b^n$ cannot be recognized by a normal FA but can be recognized by a PDA.

Simple Example / Interview Tip

If the interviewer asks for one example, say: "A DFA cannot remember an arbitrary number of a's, but a PDA can push one stack symbol for each a and later pop one for each b."

2. Define the components of a PDA formally.

Interview Answer

A PDA is formally represented as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Q is the finite set of states.
Σ is the input alphabet.
Γ is the stack alphabet.
δ is the transition function.
q_0 is the initial state.
Z_0 is the initial stack symbol.
F is the set of accepting states.

Key Points to Remember

- Q – states

- Σ – input symbols
- Γ – stack symbols
- δ – transition function
- q_0 – initial state
- Z_0 – initial stack symbol
- F – final states

3. What is a stack in PDA? Why is it important?

Interview Answer

A stack is the extra memory used by a PDA. It follows the LIFO principle, which means Last In, First Out. A symbol pushed last is the first symbol that can be popped.

The stack is important because it allows a PDA to remember an unbounded amount of information in a restricted way. For example, while reading a's in a^nb^n, the PDA pushes one symbol for every a and later pops one symbol for every b.

Key Points to Remember

- LIFO memory structure.
- Push operation adds a symbol.
- Pop operation removes the top symbol.
- Useful for matching counts and nested structures.
- Makes PDA more powerful than FA.

4. What is an Instantaneous Description (ID) in PDA?

Interview Answer

An Instantaneous Description, or ID, represents the current configuration of a PDA at a particular moment.

It is commonly written as (q, w, α), where q is the current state, w is the unread input, and α is the current stack contents.

For example, (q1, bbb, XXZ_0) means the PDA is in q1, bbb remains to be read, and XXZ_0 is on the stack.

Key Points to Remember

- Shows current state.
- Shows remaining input.
- Shows current stack contents.
- Used to describe step-by-step PDA computation.

5. What are the different types of moves in a PDA?

Interview Answer

The main PDA moves are input-consuming moves and ε-moves.

In an input-consuming move, the PDA reads one input symbol and may push or pop a stack symbol. In an ε-move, the PDA changes state and/or stack without consuming an input symbol.

Typical stack actions are push, pop, and sometimes leave the stack unchanged.

Key Points to Remember

- Input move: consumes one input symbol.
- ε-move: consumes no input.
- Push: add a stack symbol.
- Pop: remove the top stack symbol.
- No-change operation: keep the stack unchanged.

6. What is acceptance by final state and empty stack?

Interview Answer

There are two common PDA acceptance methods.

Acceptance by final state means the entire input is consumed and the PDA enters a state belonging to the final-state set F.

Acceptance by empty stack means the entire input is consumed and the stack is emptied according to the PDA's transitions.

For nondeterministic PDAs, these two methods have the same language-recognition power.

Key Points to Remember

- Final state checks the ending state.
- Empty-stack acceptance checks stack completion.
- Both characterize the context-free languages for NPDAs.
- The transition design must correctly enforce the intended condition.

Simple Example / Interview Tip

Remember: final-state acceptance checks the state; empty-stack acceptance checks the stack. For NPDAs, both have the same expressive power.

7. Define the language accepted by a PDA.

Interview Answer

The language accepted by a PDA is the set of all input strings for which the PDA has an accepting computation.

For final-state acceptance:

$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \text{ by reaching a final state after consuming } w \}$.

For a nondeterministic PDA, a string is accepted if at least one valid computation path reaches acceptance.

Key Points to Remember

- Language is a set of accepted input strings.
- Input must belong to the PDA's alphabet.
- Acceptance depends on the selected acceptance method.
- Nondeterminism means one successful path is enough.

8. How does a PDA process a string using IDs?

Interview Answer

A PDA processes a string by repeatedly changing its instantaneous description.

For example, for the language $a^n b^n$, the PDA pushes X while reading a's and pops X while reading b's.

Key Points to Remember

- Start with the initial state and initial stack.
- Read an input symbol or take an ε-move.
- Apply the corresponding transition.
- Update the state and stack.
- Repeat until the input is consumed.
- Check the acceptance condition.

9. Differentiate between Deterministic PDA and Non-Deterministic PDA.

Interview Answer

A Deterministic PDA, or DPDA, has at most one possible move for a given configuration. An NPDA can have multiple possible moves and can choose among different computation paths.

Every DPDA is an NPDA, but NPDAs are more powerful than DPDAs in language recognition because deterministic context-free languages form a proper subset of context-free languages.

Key Points to Remember

- DPDA: at most one valid move.
- NPDA: may have multiple possible moves.
- NPDA can use nondeterministic choices.
- Every DPDA language is context-free.
- Not every context-free language is deterministic context-free.

10. Why are PDAs equivalent to Context-Free Grammars (CFG)?

Interview Answer

PDAs and CFGs are equivalent in expressive power because they both characterize exactly the class of context-free languages.

A CFG can be converted into an equivalent NPDA by using the stack to simulate grammar derivations. Conversely, a PDA can be converted into an equivalent CFG using variables that represent stack-related computations.

Key Points to Remember

- CFG → equivalent NPDA.
- PDA → equivalent CFG.
- Both describe context-free languages.
- This equivalence is important in parsing and formal-language theory.

Simple Example / Interview Tip

Short answer: “CFG and NPDA are equivalent because both generate or recognize exactly the context-free languages.”

11. What types of languages are accepted by PDA? Give examples.

Interview Answer

A PDA accepts context-free languages.

Examples include:
$L_1 = \{a^n b^n \mid n \geq 1\}$
L_2 = balanced parentheses
$L_3 = \{wcw^R \mid w \in \{a,b\}^*\}$

These languages require memory that a finite automaton does not have.

Key Points to Remember

- Balanced parentheses.
- $a^n b^n$.
- Palindrome-like languages with a center marker.
- General context-free languages.

12. What is a Turing Machine? How is it more powerful than PDA?

Interview Answer

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

Interview Answer

A Pushdown Automaton, or PDA, is a finite automaton with an additional stack. The stack gives the PDA extra memory, so it can recognize many context-free languages.

A Finite Automaton has only a finite number of states and no stack. Therefore, it can recognize only regular languages. A PDA has a stack and can recognize context-free languages such as $\{a^n b^n \mid n \geq 1\}$.

Key Points to Remember

- FA has finite control only.
- PDA has finite control plus one stack.
- FA recognizes regular languages.
- PDA recognizes context-free languages.
- Example: $a^n b^n$ cannot be recognized by a normal FA but can be recognized by a PDA.

Simple Example / Interview Tip

If the interviewer asks for one example, say: “A DFA cannot remember an arbitrary number of a's, but a PDA can push one stack symbol for each a and later pop one for each b.”

2. Define the components of a PDA formally.

Interview Answer

A PDA is formally represented as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Q is the finite set of states.
Σ is the input alphabet.
Γ is the stack alphabet.
δ is the transition function.
q_0 is the initial state.
Z_0 is the initial stack symbol.
F is the set of accepting states.

Key Points to Remember

- Q – states
- Σ – input symbols
- Γ – stack symbols
- δ – transition function
- q_0 – initial state
- Z_0 – initial stack symbol
- F – final states

3. What is a stack in PDA? Why is it important?

Interview Answer

A stack is the extra memory used by a PDA. It follows the LIFO principle, which means Last In, First Out. A symbol pushed last is the first symbol that can be popped.

The stack is important because it allows a PDA to remember an unbounded amount of information in a restricted way. For example, while reading a's in a^nb^n, the PDA pushes one symbol for every a and later pops one symbol for every b.

Key Points to Remember

- LIFO memory structure.
- Push operation adds a symbol.
- Pop operation removes the top symbol.
- Useful for matching counts and nested structures.
- Makes PDA more powerful than FA.

4. What is an Instantaneous Description (ID) in PDA?

Interview Answer

An Instantaneous Description, or ID, represents the current configuration of a PDA at a particular moment.

It is commonly written as (q, w, α), where q is the current state, w is the unread input, and α is the current stack contents.

For example, (q1, bbb, XXZ_0) means the PDA is in q1, bbb remains to be read, and XXZ_0 is on the stack.

Key Points to Remember

- Shows current state.
- Shows remaining input.
- Shows current stack contents.
- Used to describe step-by-step PDA computation.

5. What are the different types of moves in a PDA?

Interview Answer

The main PDA moves are input-consuming moves and ε-moves.

In an input-consuming move, the PDA reads one input symbol and may push or pop a stack symbol. In an ε-move, the PDA changes state and/or stack without consuming an input symbol.

Typical stack actions are push, pop, and sometimes leave the stack unchanged.

Key Points to Remember

- Input move: consumes one input symbol.
- ε-move: consumes no input.
- Push: add a stack symbol.
- Pop: remove the top stack symbol.
- No-change operation: keep the stack unchanged.

6. What is acceptance by final state and empty stack?

Interview Answer

There are two common PDA acceptance methods.

Acceptance by final state means the entire input is consumed and the PDA enters a state belonging to the final-state set F.

Acceptance by empty stack means the entire input is consumed and the stack is emptied according to the PDA's transitions.

For nondeterministic PDAs, these two methods have the same language-recognition power.

Key Points to Remember

- Final state checks the ending state.
- Empty-stack acceptance checks stack completion.
- Both characterize the context-free languages for NPDAs.
- The transition design must correctly enforce the intended condition.

Simple Example / Interview Tip

Remember: final-state acceptance checks the state; empty-stack acceptance checks the stack. For NPDAs, both have the same expressive power.

7. Define the language accepted by a PDA.

Interview Answer

The language accepted by a PDA is the set of all input strings for which the PDA has an accepting computation.

For final-state acceptance:

$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \text{ by reaching a final state after consuming } w \}$.

For a nondeterministic PDA, a string is accepted if at least one valid computation path reaches acceptance.

Key Points to Remember

- Language is a set of accepted input strings.
- Input must belong to the PDA's alphabet.
- Acceptance depends on the selected acceptance method.
- Nondeterminism means one successful path is enough.

8. How does a PDA process a string using IDs?

Interview Answer

A PDA processes a string by repeatedly changing its instantaneous description.

For example, for the language $a^n b^n$, the PDA pushes X while reading a's and pops X while reading b's.

Key Points to Remember

- Start with the initial state and initial stack.
- Read an input symbol or take an ε-move.

- Apply the corresponding transition.
- Update the state and stack.
- Repeat until the input is consumed.
- Check the acceptance condition.

9. Differentiate between Deterministic PDA and Non-Deterministic PDA.

Interview Answer

A Deterministic PDA, or DPDA, has at most one possible move for a given configuration. An NPDA can have multiple possible moves and can choose among different computation paths.

Every DPDA is an NPDA, but NPDAs are more powerful than DPDAs in language recognition because deterministic context-free languages form a proper subset of context-free languages.

Key Points to Remember

- DPDA: at most one valid move.
- NPDA: may have multiple possible moves.
- NPDA can use nondeterministic choices.
- Every DPDA language is context-free.
- Not every context-free language is deterministic context-free.

10. Why are PDAs equivalent to Context-Free Grammars (CFG)?

Interview Answer

PDAs and CFGs are equivalent in expressive power because they both characterize exactly the class of context-free languages.

A CFG can be converted into an equivalent NPDA by using the stack to simulate grammar derivations. Conversely, a PDA can be converted into an equivalent CFG using variables that represent stack-related computations.

Key Points to Remember

- CFG → equivalent NPDA.
- PDA → equivalent CFG.
- Both describe context-free languages.
- This equivalence is important in parsing and formal-language theory.

Simple Example / Interview Tip

Short answer: “CFG and NPDA are equivalent because both generate or recognize exactly the context-free languages.”

11. What types of languages are accepted by PDA? Give examples.

Interview Answer

A PDA accepts context-free languages.

Examples include:

$L_1 = \{a^n b^n \mid n \geq 1\}$

L_2 = balanced parentheses

$L_3 = \{wcw^R \mid w \in \{a,b\}^*\}$

These languages require memory that a finite automaton does not have.

Key Points to Remember

- Balanced parentheses.
- $a^n b^n$.
- Palindrome-like languages with a center marker.
- General context-free languages.

12. What is a Turing Machine? How is it more powerful than PDA?

Interview Answer

A Turing Machine is a mathematical model of general computation. It uses an unbounded tape, a read/write head and a finite control.

It is more powerful than a PDA because it has read/write tape memory and can normally move the head both left and right. A PDA has only one stack with LIFO access, so it cannot recognize every language that a Turing Machine can.

Key Points to Remember

- TM has unbounded read/write tape.
- TM head can move left and right in the standard model.
- PDA has one stack.
- TM can solve problems beyond context-free languages.
- Example: $a^n b^n c^n$ can be recognized by a TM but not by a PDA.

Simple Example / Interview Tip

A good example is $a^n b^n c^n$: a PDA cannot recognize it, while a Turing Machine can.

13. Define the components of a Turing Machine.

Interview Answer

A standard Turing Machine can be represented as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Q is the set of states.
Σ is the input alphabet.
Γ is the tape alphabet.
δ is the transition function.
q_0 is the initial state.

B is the blank symbol.
F is the set of accepting states.

Key Points to Remember

- Q – states
- Σ – input alphabet
- Γ – tape alphabet
- δ – transition function
- q_0 – initial state
- B – blank symbol
- F – accepting states

14. Explain tape, head, and state transitions in a Turing Machine.

Interview Answer

The tape is the machine's unbounded storage area and is divided into cells. The head reads a symbol from a cell, can write a symbol, and moves left or right.

A transition is commonly written as $\delta(q, a) = (p, b, D)$, meaning that in state q reading a, the machine writes b, changes to state p, and moves in direction D.

Key Points to Remember

- Tape stores input and intermediate information.
- Head reads and writes.
- State represents the current control condition.
- Transition specifies write, next state and movement.
- Standard directions are Left and Right.

15. Explain programming techniques for Turing Machines.

Interview Answer

Turing Machines can be designed using programming techniques that make large transition systems easier to understand.

Common techniques include marking symbols, using delimiters, scanning left or right to find symbols, repeated passes, storing limited information in states, and designing conceptual subroutines.

Key Points to Remember

- Mark processed symbols using X, Y, etc.
- Use delimiters to separate sections.
- Use repeated scans.
- Use states as limited finite memory.
- Break a complex task into conceptual subroutines.
- Return to a known tape position between operations.

16. What is storage in finite control in Turing Machines?

Interview Answer

Storage in finite control means keeping a small amount of information in the current state of the machine.

For example, a state can remember that the machine has just seen a particular symbol or is currently in a particular phase of an algorithm. However, finite control has only a finite number of states, so it cannot store an arbitrarily long string.

Key Points to Remember

- State can remember a finite condition.
- Useful for flags and phases.
- Finite control is limited.
- Unbounded information must be stored on the tape.

17. Compare FA, PDA, and TM based on memory, power, and language acceptance.

Interview Answer

The three models form an increasing hierarchy of computational power.

FA has finite memory and recognizes regular languages.
PDA has a stack and recognizes context-free languages.
TM has an unbounded read/write tape and supports general computation.

Key Points to Remember

- FA: finite control; regular languages.
- PDA: one stack; context-free languages.
- TM: unbounded tape; much more general computation.
- FA < PDA < TM in computational power.

Simple Example / Interview Tip

For a quick comparison, remember: FA = finite memory, PDA = stack, TM = unbounded tape.

18. What is DFA?

Interview Answer

DFA stands for Deterministic Finite Automaton. It is a finite automaton in which for every state and input symbol, exactly one next state is defined.

A DFA has no stack or other unbounded memory. It recognizes regular languages.

Key Points to Remember

- DFA = Deterministic Finite Automaton.
- Exactly one transition for each state/input pair.
- No stack.
- Recognizes regular languages.
- Used for lexical analysis and pattern recognition.

19. Which language is accepted by a Turing Machine?

Interview Answer

A Turing Machine can accept Turing-recognizable languages. If the machine is guaranteed to halt on every input, it decides a decidable language.

Therefore, Turing Machines are capable of recognizing many languages beyond regular and context-free languages.

Key Points to Remember

- Regular languages can be recognized.
- Context-free languages can be recognized.
- Many non-context-free languages can be recognized.
- Turing-recognizable languages may contain machines that do not halt on rejected inputs.
- Decidable languages are those for which a TM always halts and gives the correct answer.

20. How are Turing Machines applied in real-world applications?

Interview Answer

Turing Machines are mainly theoretical models rather than machines directly deployed in ordinary applications. Their importance is that they provide a foundation for understanding algorithms and computation.

The ideas behind Turing Machines are reflected in compilers, interpreters, programming languages, algorithm design, formal verification, computability theory and computer architecture.

Key Points to Remember

- Foundation for computation theory.
- Helps analyze algorithmic computability.
- Supports formal verification and language theory.
- Conceptually related to general-purpose computation.
- Useful for understanding limits of what computers can compute.

QUICK REVISION TABLE

No.	Topic	One-line answer
1	PDA vs FA	PDA has a stack; FA has only finite control.
2	PDA components	7-tuple: Q, Σ, Γ, δ, q_0, Z_0, F.
3	Stack	LIFO memory used for matching/nesting.
4	ID	Current state + unread input + stack.
5	PDA moves	Input moves and ε-moves; push/pop/no-change.
6	Acceptance	Final state or empty stack.
7	PDA language	Set of strings having an accepting PDA computation.
8	IDs	Successive configurations show how the PDA processes input.
9	DPDA vs NPDA	DPDA is deterministic; NPDA can have multiple choices.
10	PDA = CFG	Both characterize context-free languages.
11	PDA languages	Context-free languages such as $a^n b^n$.
12	TM vs PDA	TM has more general read/write tape memory.
13	TM components	7-tuple with tape alphabet and blank symbol.
14	Tape/head	Head reads, writes and moves left/right.
15	TM techniques	Marking, delimiters, scans and subroutines.
16	Finite control	Stores only finite information in states.
17	FA/PDA/TM	Finite control / stack / unbounded tape.
18	DFA	Deterministic finite automaton.
19	TM languages	Turing-recognizable; decidable if it always halts.
20	Real-world role	Foundation for algorithms, computability and formal verification.

HOW TO ANSWER IN THE INTERVIEW

- Start with a one-sentence definition.
- Then give two or three important points.
- Give one simple example when possible.
- Use correct terms such as stack, transition function, context-free language, tape and finite control.
- If asked for a comparison, use a small table or explain the key difference directly.
- Do not overcomplicate the answer. The uploaded assessment is a one-hour, 20-question mock interview, so answers should be clear and concise. □filecite□turn10file0□L9-L11□

RUBRIC ALIGNMENT

The document is designed to support the assessment rubric's requirements for conceptual understanding, accuracy, application of knowledge, completeness and clarity/presentation. **Key Points to Remember**

- TM has unbounded read/write tape.
- TM head can move left and right in the standard model.
- PDA has one stack.
- TM can solve problems beyond context-free languages.
- Example: $a^nb^nc^n$ can be recognized by a TM but not by a PDA.

Simple Example / Interview Tip

A good example is $a^nb^nc^n$: a PDA cannot recognize it, while a Turing Machine can.

13. Define the components of a Turing Machine.

Interview Answer

A standard Turing Machine can be represented as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Q is the set of states.
Σ is the input alphabet.
Γ is the tape alphabet.
δ is the transition function.
q_0 is the initial state.
B is the blank symbol.
F is the set of accepting states.

Key Points to Remember

- Q – states
- Σ – input alphabet
- Γ – tape alphabet
- δ – transition function
- q_0 – initial state
- B – blank symbol
- F – accepting states

14. Explain tape, head, and state transitions in a Turing Machine.

Interview Answer

The tape is the machine's unbounded storage area and is divided into cells. The head reads a symbol from a cell, can write a symbol, and moves left or right.

A transition is commonly written as $\delta(q, a) = (p, b, D)$, meaning that in state q reading a, the machine writes b, changes to state p, and moves in direction D.

Key Points to Remember

- Tape stores input and intermediate information.
- Head reads and writes.
- State represents the current control condition.
- Transition specifies write, next state and movement.
- Standard directions are Left and Right.

15. Explain programming techniques for Turing Machines.

Interview Answer

Turing Machines can be designed using programming techniques that make large transition systems easier to understand.

Common techniques include marking symbols, using delimiters, scanning left or right to find symbols, repeated passes, storing limited information in states, and designing conceptual subroutines.

Key Points to Remember

- Mark processed symbols using X, Y, etc.
- Use delimiters to separate sections.
- Use repeated scans.
- Use states as limited finite memory.
- Break a complex task into conceptual subroutines.
- Return to a known tape position between operations.

16. What is storage in finite control in Turing Machines?

Interview Answer

Storage in finite control means keeping a small amount of information in the current state of the machine.

For example, a state can remember that the machine has just seen a particular symbol or is currently in a particular phase of an algorithm. However, finite control has only a finite number of states, so it cannot store an arbitrarily long string.

Key Points to Remember

- State can remember a finite condition.
- Useful for flags and phases.
- Finite control is limited.
- Unbounded information must be stored on the tape.

17. Compare FA, PDA, and TM based on memory, power, and language acceptance.

Interview Answer

The three models form an increasing hierarchy of computational power.

FA has finite memory and recognizes regular languages.
PDA has a stack and recognizes context-free languages.
TM has an unbounded read/write tape and supports general computation.

Key Points to Remember

- FA: finite control; regular languages.
- PDA: one stack; context-free languages.
- TM: unbounded tape; much more general computation.
- FA < PDA < TM in computational power.

Simple Example / Interview Tip

For a quick comparison, remember: FA = finite memory, PDA = stack, TM = unbounded tape.

18. What is DFA?

Interview Answer

DFA stands for Deterministic Finite Automaton. It is a finite automaton in which for every state and input symbol, exactly one next state is defined.

A DFA has no stack or other unbounded memory. It recognizes regular languages.

Key Points to Remember

- DFA = Deterministic Finite Automaton.
- Exactly one transition for each state/input pair.
- No stack.
- Recognizes regular languages.
- Used for lexical analysis and pattern recognition.

19. Which language is accepted by a Turing Machine?

Interview Answer

A Turing Machine can accept Turing-recognizable languages. If the machine is guaranteed to halt on every input, it decides a decidable language.

Therefore, Turing Machines are capable of recognizing many languages beyond regular and context-free languages.

Key Points to Remember

- Regular languages can be recognized.
- Context-free languages can be recognized.
- Many non-context-free languages can be recognized.
- Turing-recognizable languages may contain machines that do not halt on rejected inputs.
- Decidable languages are those for which a TM always halts and gives the correct answer.

20. How are Turing Machines applied in real-world applications?

Interview Answer

Turing Machines are mainly theoretical models rather than machines directly deployed in ordinary applications. Their importance is that they provide a foundation for understanding algorithms and computation.

The ideas behind Turing Machines are reflected in compilers, interpreters, programming languages, algorithm design, formal verification, computability theory and computer architecture.

Key Points to Remember

- Foundation for computation theory.
- Helps analyze algorithmic computability.
- Supports formal verification and language theory.
- Conceptually related to general-purpose computation.
- Useful for understanding limits of what computers can compute.

QUICK REVISION TABLE

No.	Topic	One-line answer
1	PDA vs FA	PDA has a stack; FA has only finite control.
2	PDA components	7-tuple: Q, Σ, Γ, δ, q_0, Z_0, F.
3	Stack	LIFO memory used for matching/nesting.
4	ID	Current state + unread input + stack.
5	PDA moves	Input moves and ε-moves; push/pop/no-change.
6	Acceptance	Final state or empty stack.
7	PDA language	Set of strings having an accepting PDA computation.
8	IDs	Successive configurations show how the PDA processes input.
9	DPDA vs NPDA	DPDA is deterministic; NPDA can have multiple choices.
10	PDA = CFG	Both characterize context-free languages.
11	PDA languages	Context-free languages such as $a^n b^n$.
12	TM vs PDA	TM has more general read/write tape memory.
13	TM components	7-tuple with tape alphabet and blank symbol.
14	Tape/head	Head reads, writes and moves left/right.
15	TM techniques	Marking, delimiters, scans and subroutines.
16	Finite control	Stores only finite information in states.
17	FA/PDA/TM	Finite control / stack / unbounded tape.
18	DFA	Deterministic finite automaton.
19	TM languages	Turing-recognizable; decidable if it always halts.
20	Real-world role	Foundation for algorithms, computability and formal verification.